



paperRをつかってdfのラベル管理をしたいと思います。

理由：mutateやjoinなど、型の不一致で発生するエラーが多すぎるため、基本的にラベル属性は外して作業を進めたい

これに合わせて関連の関数群を作成したいと思います

- ・（ラベル付き変数：numeric/character）元データは主にspssなので、一旦データを読み込んだ後でpaperRのラベルをセット。その後オリジナルのラベル属性を削除
- ・（FACTOR）paperRでファクターの扱いがどうなっているのかよくわかりません

作成したい関数(他にどのような関数を用意すべきでしょうか)

- ・ spssデータ読み込み→paperR変換矢印ラベル属性削除
- ・ paperRラベル→spss形式に戻す
- ・ ファクターの扱い→不明

利用を想定する主なパッケージ

- ・ dplyr/tidyverse系
- ・ ggplot2
- ・ gt-table
- ・ svydesign(データ構造がcluster randomised sampling/repeated cross section)

まず前提を一つだけ整理すると、ここで使いたいのは「paperR」ではなく、CRANにある「paperR」パッケージ（変数ラベル機能をもつやつ）ですよね。^{[1] [2]}

以下、「SPSS→paperR→dplyr/ggplot2→SPSS」という一連の流れを想定してアイデアを整理します。

1. paperR のラベルの基本

- `labels(data)` でデータフレーム全体の変数ラベルが取れる（名前付き character ベクタ）。^[3]
^[1]
- `labels(data) <- c(...)` で一括セットでき、`labels(data, which = "var") <- "ラベル"` で個別に設定できる。^[3]
- ラベルを付けたデータは `is.labeled(data)` で「labelled data.frame」として認識される。^[3]

ここまではご存じかもしれませんが、「ラベル情報は単なる属性」なので、mutate/join 前に属性を落としてしまう方針は自然です。^[1]

2. 想定すると便利そうな関数群

(A) SPSS 読み込み → paperR ラベルに変換 → 元の属性削除

前提として:

- 読み込みには `haven::read_sav()` や `read_spss()` を使う
- 変数ラベルは `haven::labelled` の属性 (`label`) として付いている
- 値ラベルは `haven::labelled` の `labels` 属性にある

という典型パターンを想定します。^{[4] [5]}

作りたい関数のイメージ:

1. `haven::read_sav()` で取り込み
2. 各変数について
 - 変数ラベルを取り出し、ベクタにまとめる
 - 値ラベルはあとで `factor` にするために別オブジェクトとして保存するか、その場で `factor` 化する
3. `labels(df) <- var_label_vector` で paperR のラベルをセット
4. `attr(df[[var]], "label")` や `class(df[[var]]) <- setdiff(class(df[[var]]), "labelled")` などで `haven` の属性を削除
(あるいは `zap_labels()` のような関数を自作)

ここで一つ設計ポイントは、「値ラベルをどこまで保存しておきたいか」です。

- 「解析中は全部 `numeric` にしてラベルは別メタデータで管理」で割り切るか
- 「カテゴリ変数は `factor` にして `level` にラベルを入れる」かのどちらかを関数側で決めておくとな後が楽です。^{[6] [4]}

(B) paperR ラベル → SPSS 形式に戻す

「SPSS 形式に戻す」には最低でも:

- 変数ラベル
- 値ラベル (必要なら)

を `haven::labelled` 形式に戻す必要があります。^{[5] [4]}

関数イメージ:

1. `var_labs <- labels(df)` で変数ラベルを取得
2. 変数ラベルがある変数について `attr(df[[var]], "label") <- var_labs[var]` を付与
3. カテゴリ変数 (`factor`) については、`levels()` と `labels()` から「コード値→ラベル」の対応表を作り、`haven::labelled()` で `labels` 属性として付加
4. `haven::write_sav()` などで SPSS ファイルを書き出し

値ラベルについては、内部コードを numeric にするか character にするかをどこかで一意に決めておく必要があります（SPSS と往復するときにズレやすいところです）。^[4]

3. papeR における factor の扱い

papeR のラベル機能は基本的に「変数ラベル」にフォーカスしていて、factor の level ラベルは通常の R の factor 機能に任せています。^{[1] [3]}

- factor 自体は普通の factor クラスのまま
- table() や latex.table() 系の関数で、factor の level 名（カテゴリ名）を使ったクロス表などをきれいに出す、という使い方が想定されています。^[3]

したがって、

- SPSS の「値ラベル」を R でどう扱うか
 - factor の levels に直接入れる
 - あるいは別メタ (list) として持ち、必要なときだけ factor に変換

を事前の方針決めしておく、と papeR 側はシンプルに「変数ラベルだけ扱う」運用ができます。^{[6] [4]}

4. 追加で用意しておく便利な関数

ユーザー側の作業負担を減らすという観点で、次のような補助関数も有用だと思います。

- strip_all_labels(df)
 - データから label 属性や labels 属性、labelled クラスを一括削除 (dplyr 前に使う)
- restore_paper_labels(df, meta)
 - 一度保存しておいたラベル情報 (変数ラベル + 値ラベル) を、加工後の df に再アタッチ
 - meta は「変数名ごとのラベル情報」をまとめた data.frame や list
- factor_from_labels(df, vars, numeric_codes = FALSE)
 - 指定した変数を factor 化 (SPSS の値ラベルを level に)
 - numeric_codes = TRUE ならコード値を integer、ラベルは level 名にするなど
- svydesign_with_labels(...)
 - svydesign() で作ったオブジェクトの中のデータに対し、元の df からラベルをコピーするラッパ関数

dplyr や ggplot2、gt、svydesign は、基本的に「型が素直であればラベルは無視してくれる」ので、

- 「解析用 df は属性をすべて落としたきれいな tibble」
- 「ラベルは別メタか、papeR の labels() だけに集約」

という設計にしておく、今回の「型不一致でのエラー」はかなり減らせるはずです。^{[7] [1]}

ここまですを踏まえて、最初の実装してみるとしたら、

「`read_spss_clean()` (仮称) : SPSS 読み込み→papeR ラベルに変換→haven 属性削除」
から着手するのが良さそうに思います。

どの方向性で factor (値ラベル) を扱いたいのか、

- 解析中も factor で持ちたい
- 解析中は numeric にして、集計や出力時だけ factor にする

のどちらが、先生の現在のプロジェクトのワークフローにはしっくりきそうでしょうか？

森

papeRの思想に合わせて、factorはそのままにする が、numericとして扱うか、characterとして扱うか 統一した方が良いでしょうか？

文字列ベースのfactor変数もデータフレームにあるので、characterにした方が良いのか？
後で分析に使う際、レベルと実際の値が混乱することがあります

結論から言うと、

- 「カテゴリ変数として分析に使うもの」は **factor** のまま
- 「自由記述などテキストとして扱うもの」は **character** のまま
- どちらも「numeric に見えるけど本質的にはカテゴリ」のものは **numeric** では統一しない

という方針の方が、papeR の設計とも一般的な R の流儀とも整合的です。 [16] [17]

なぜ numeric 統一はおすすめしないか

- factor は内部的には整数コード (1, 2, 3, ...) ですが、これはレベルを識別するためであって、**量的な意味のある数値ではない**からです。 [17] [18]
- numeric にしてしまうと、`mutate` 中で普通に数値演算が可能になり、誤って平均や和を取ってしまうなど「間違ってもエラーにならない」危険が出ます。 [19] [17]
- 回帰や `svyglm` などでも、「本来ダミー化すべきカテゴリ」を誤って連続変数として扱ってしまうリスクがあります。 [17]

character に統一すべきか？

- 文字列ベースのカテゴリ (例: 地名、市区町村名) は、
 - 「集計・モデリングでカテゴリとして使う」→ factor
 - 「テキスト情報として表示だけに使う」→ character
で分けるのが一般的です。 [18] [17]
- すべて character にしてしまうと、

- モデルや `summarize_factor()` 系の関数が「カテゴリ変数」と認識してくれず、毎回 `as.factor()` が必要になる
- p 値付きのカテゴリ別サマリーなど、papeR の「factor 用便利機能」が活きないというデメリットがあります。 [20] [16]

混乱を減らすための「運用ルール」の提案

レベルと実際の値が混乱しないように、次のようなルールをおすすめします。

1. 「量的」か「質的」かでまず分ける

- 量的（年齢、身長、所得など）→ `numeric`
- 質的（性別、教育レベル、回答カテゴリなど）→ `factor`

2. SPSS の値ラベル付き `numeric` 変数は、R では「コード値 + `factor` レベル名」ではっきり分ける

- 例： `sex_code` (1, 2) , `sex` (`factor`: "Male", "Female") のように、コード (`numeric`) とラベル (`factor`) を別カラムに分離する運用にするとかなり混乱が減ります。 [21] [22]

3. 文字列ベースのカテゴリは、早めに `factor` に変換しておく

- 例： `region` が文字列なら `region <- factor(region)`
- 解析に使わないフリーテキストだけ `character` のまま残す。 [18] [17]

4. papeR には「`numeric` と `factor`」を渡すことを前提にする

- `summarize()` や `summarize_factor()` などは `numeric` と `factor` を想定しているので、その2種類に整理しておくと、レポート生成時の挙動も予測しやすくなります。 [16] [20]

practical にやるなら

- SPSS 読み込み直後に、
 - 「カテゴリとして使う変数名のリスト」を作る（例： `cat_vars <- c("sex", "edu", "region", ...)`）
 - そこだけ `as_factor()` → `factor` か、コードとラベルの二重構造に変換
 - それ以外は `numeric` / `character` に整理
という「型リセット用スクリプト or 関数」を一発かませておくと、その後の `dplyr` / `svydesign` / `papeR` がずっと安定します。 [23] [24]

papeR の文脈に合わせて言うと、「`factor` は `factor` のまま」「`numeric` に本質的な意味があるものだけ `numeric`」「それ以外は `character`」と三分する設計が一番きれいだと思います。 [25] [16]

先生の今のデータで、**「コード値 (1,2,3...) とラベルが特に混乱しやすい変数」**って具体的にどのあたりでしょうか？ その1・2例を挙げていただければ、型の決め方をもう少し具体的に提案できます。

添付のとおりfactor変数が多いのですが、 is.numeric も is.numeric も is.logical も false という変数 も多いです

添付ファイルを見ると、「factor だが is.numeric も is.character も is.logical も FALSE」というのは、典型的な haven 由来の labelled 変数や factor 変数で、内部コードが 1/2 などの整数になっているケースだと思います。^[32]

この状況に対しては、次のように割り切ると整理しやすいです。

1. このタイプの変数の位置づけ

例に挙げられている変数 (nt_ch_sev_stunt, rh_anc_iron, rc_agri など) は、

- `is_factor == TRUE` かつ `is_binary == TRUE`
- `factor_levels` は "Yes_No", "No_Yes", "Non-Agriculture_Agriculture" など
- `var1~var5` は 1/2 のコード値

という構造なので、「コード値 1/2 を内部に持つ factor (二値変数)」と解釈するのが自然です。^[32]

これらは、`is.numeric` や `is.character` が FALSE なのは正常で、「カテゴリ変数として扱うべき」対象です。

2. どう型を統一するか (実務的な方針)

このような変数については、次のルールをおすすめします。

1. factor のまま使う (numeric にはしない)

- 1/2 のコード値は「意味のある量」ではないので、numeric として扱うと回帰や集計で誤解を生みやすいです。
- R の側から見れば「二値の factor」としてロジスティック回帰や `svyglm` のダミー化などに素直に利用できます。

2. コード値を残すかどうかを変数ペアで分離する

- どうしても「元の SPSS のコード値 (1=No, 2=Yes など)」を残しておきたい場合は、
 - `xxx_code` (numeric: 1/2) ,
 - `xxx` (factor: "No"/"Yes")のようにコード用とカテゴリ用を別カラムに分けるのが安全です。^[32]

3. character への一括変換は避ける

- すべて character にしてしまうと、`dplyr/ggplot2` だけでなく `svydesign` や `glm` などで毎回 `as.factor()` が必要になり、`papeR` の factor 用の便利機能も活かしにくくなります。^[33]
- 「自由記述」「ID 文字列」のような、統計的にはカテゴリとして扱わないものだけ character にしておく、という分け方がよいです。^{[33] [32]}

3. 混乱を減らすための具体的な整理手順

添付ファイルのような summary を起点に、以下のような処理を一度かけてしまう関数を用意しておくのと楽です。

1. 二値 factor の整理

- `is_factor == TRUE & is_binary == TRUE` の変数を抽出。^[32]
- それらについて
 - 解析用には factor のまま
 - 必要なら `xxx_code <- as.integer(xxx)` でコード値を別カラムにコピー

2. 多値 factor (`is_factor == TRUE & !is_binary`) も同様に「factor + (任意で) コード」へ整理

3. `is_factor == FALSE` かつ `is_numeric == FALSE` かつ `is_character == FALSE` な変数

- ここが「中途半端な labelled」や「変なクラス」を持っている候補なので、
 - `class()` を確認し、labelled など余計なクラスを落として numeric か factor に正規化
 - もしくは、あらかじめ「カテゴリとして使う変数一覧」「連続として使う変数一覧」を作って、それに従って型を決める

4. papeR との整合性

papeR 自体は、

- 数量データは numeric
- カテゴリデータは factor
- ラベルは `labels()` による「変数ラベル」を中心に管理

という思想なので、今回のような Yes/No 系の変数は、「factor を基本型」として整理しておくのが自然です。^{[34] [35] [32]}

もしよろしければ、

- 「`is_factor` は FALSE だが、`is_numeric` も `is_character` も FALSE」という具体的な変数名を 2〜3 個挙げていただければ、そのケースに対して「どうキャストするか」のサンプルコードを書いてみます。

✻

すみません、私の勘違いで、is_factorがfalseの場合は、必ずcharacter/numeric/binaryのいずれかがtrueになっていました

なるほど、状況整理ありがとうございます。勘違いが解けたということは、「少なくともどの変数も

- factor
- もしくは numeric / character / logical

のいずれかにはきちんと分類されている」ということですね。^[36]

この前提なら、型設計としてはかなりシンプルにできます。

いま分かったことから言えること

- `is_factor == TRUE` な変数は、すべて「カテゴリ変数」として扱う候補でよい。^[36]
- `is_factor == FALSE` で `is_numeric == TRUE` なものは、基本的に「量的変数」扱い。
- `is_factor == FALSE` で `is_character == TRUE` なものは、「ID や自由記述、ラベル文字列」扱いにできる。
- 「型がよく分からないグレーゾーン」は実は存在していなかったのもので、
 - あとは「カテゴリとして使いたい numeric / character を factor に昇格させるかどうか」だけを決めればよい。^[36]

今後の運用のイメージ（整理版）

1. 読み込み直後に一括で型を整えるレイヤーを作る

- SPSS 読み込み →
- 「この変数はカテゴリとして使う」というリストに基づき、必要なものを factor 化（または factor に変換）
- それ以外は numeric / character のまま
という「型クリーニング関数」を一度かませる。

2. papeR のラベルは、その「型を整えた後」のデータフレームに対してだけ付与する

- これで、後続の `mutate`, `join`, `svydesign` では、ラベルが原因の型エラーは起きにくくなる。

今の理解に基づくと、「一見おかしい型の変数が大量にある」という懸念は、とりあえず解消できそうです。^[36]

次の一步としては、

- 「カテゴリとして扱いたいが、今は numeric / character のままの変数」がどれくらいあるか
- それらを「どこまで factor にしておくか」（全部やるか、分析に使うものだけにするか）

を一緒に決めていくと、ワークフローが固まりやすいと思います。

いまのデータの中で、「カテゴリとして使うけれど現在 factor になっていない代表的な変数」を 1〜2 個挙げてもらえますか？それを例に、変換とラベル付けの具体的な流れを組み立ててみましょう。

✂

最終的な変数の整理として、分析作業のため、factor は全てダミー化、連続変数は numeric として、カテゴリ化に必要な変数 (state, group_treated 等) のみ factor として残そうと考えています

その方針（「基本は連続 + ダミー、state や group_treated などキーだけ factor 残し」）は、とても整理が良くて、実務的にも一貫したやり方だと思います。[37] [38]

ここでだけ注意したいポイントを3つだけ挙げます。

1. 「いつダミー化するか」を決めておく

- R の `lm`, `glm`, `svyglm` などは、**factor** を渡せば内部で自動的にダミー化します。[38] [37]
- 一方で、「自分でダミーを作っておきたい」のは
 - LASSO など、`model.matrix()` ベースの設計行列を明示的に使うとき
 - 他のソフト (Stata, SPSS, Python) に渡すときが多いと思います。[39] [40]

なので、

- モデリングのたびに `model.matrix()` で必要なダミーを作る
- あるいは「分析用マスター df」だけは、事前に全 factor をダミー化しておく

のどちらかをワークフローとして決めてしまうとブレにくいです。[41] [39]

2. `svydesign` との関係

- `svydesign()` 自体は、`strata` や `cluster`、`weights` は **factor** でも **numeric** でも扱えます。[42] [43]
- ただし、「strata や PSU の ID」は、
 - 漏れなく factor にしておく
 - もしくは character のまま ID として使うのどちらかに統一した方が、後で `subset` などするときに混乱が減ります。

先生の方針で言うと、`state`, `group_treated`, `cluster_id`, `strata_id` のような変数は **factor** のまま残すのがよさそうです。[44] [42]

3. 実装イメージ（整理レイヤー）

変数整理の最後に、1 本関数を噛ませるイメージです。

- 入力: 「整形済みの df」と「factor を残したい変数名ベクトル」
- 手順:

1. `keep_fac <- c("state", "group_treated", ...)` を定義

2. `df[setdiff(factor_vars, keep_fac)]` については

- `model.matrix(~ var - 1, data = df)` などでダミー化して列追加
- 元の factor 列を削除

3. numeric と character はそのまま

こうしておけば、**最終分析用 df は「numeric + dummy + ごく一部の factor」**という形に統一されます。^{[40] [38]}

この方針で進めるとして、次は「どの段階でダミー化するか」（SPSS から読み込んだ直後／paperR でラベルを付けた後／svydesign の前後）のどれが一番しっくり来そうでしょうか？

森

分析結果の解釈上、ファクターよりもバイナリ変数の方が解釈しやすいので。基本的にはダミー化した変数のみで分析を進める予定です

ダミー化については、読み込んだ年次ごと、分野ごとのデータを一つの大きなデータセットにまとめるので、その直後にバイナリ化して、元々のfactor変数も削除する想定です（大きなデータセットは念のため保存）

「大きなデータセットを作った直後に一括でバイナリ化して、元の factor は削除する」という設計、とても筋が良いと思います。^{[52] [53]}

この方針で重要になるのは、次の3点です。

1. 二値 factor → バイナリの変換ルールを固定する

添付の summary だと、`factor_levels` が "Yes_No" と "No_Yes" の両方が混在しています。^[52]
ここを放置したまま `as.numeric()` などをかけると、「1=Yes の変数」と「2=Yes の変数」が混在してしまい、解釈が破綻します。

なので、

- すべての二値変数について
 - 「1=基準（0 にしたい側）」 「2=1 にしたい側」など、**0/1 の定義を一意に決める**
 - 例えば「Yes 側を 1、No 側を 0」に統一
- "No_Yes" のようなレベル順の違いは、変換関数の中で自動的に吸収する

という関数を一度作っておくと安全です。^[54] ^[52]

2. 多カテゴリ factor の扱い

先生の方針だと、最終的には

- `state`, `group_treated` など「キーになる少数の factor」だけを残し
- それ以外のカテゴリ変数は、
 - 2カテゴリなら 1 本のダミー
 - 3カテゴリ以上なら $k - 1$ 本のダミー（あるいは必要に応じて全部）

という形にしておく、というイメージだと思います。^[55]

このとき、

- `model.matrix(~ var - 1)` を使うと、「レベルごとに 0/1 列」を自動生成できます。^[56] ^[55]
- ただし、「どのレベルを基準にするか」は自前で決める必要があるので、
 - 解析で使う変数だけを対象にする
 - できれば「参照カテゴリ」を変数ごとにメタデータとして保存しておく

と、後で結果の解釈や論文の表作成がかなり楽になります。

3. ワークフローとしての位置づけ

先生がお書きになったように、

1. 年次・分野ごとのデータを読み込む
2. ひとつの「大きな df」に結合（ここまでは factor もそのまま）
3. その直後に
 - バイナリ化関数を一括でかける（`*_bin` 列を作成）
 - 元の factor 列は削除（ただし「`state` など残したいもの」は除外）
4. この「分析用 df」を保存しておき、以降の `dplyr`, `svydesign`, 回帰分析はすべてこれを使う

という流れで固定すると、途中で型が揺れないので、かなり安定した運用になると思います。^[53] ^[52]

このバイナリ化の「変換ルール」を一度きっちり決めておくのが肝です。

例えば `Yes_No` / `No_Yes` / `Non-Agriculture_Agriculture` など、今見えている代表的なパターンについて、「どちらを 1 にするか」を一覧にしておくのがよさそうですが、先生の感覚だと、どの変数（例：`rc_agri` など）は「1=ベース」「1=関心側」にしたいでしょうか？

基本的な考えとしては以下のような感じです

- ・バイナリ化変数の名称に`_yes/_no`という接尾語を付与した上で、1/0の値を付与
- ・`_yes/_no`のいずれを残すかは、基本は全て`_yes`を残す
- ・したがって変数名は、オリジナル変数名+`:カテゴリラベル+_yes`とする
- ・その上で多カテゴリ変数については、すべてのバイナリダミーを残すのではなく、適宜、レベル1と2を統合など、により手作業で新変数を作成し、複数レベルをまとめたダミー変数を手作業で作成する

発想・方針はかなり一貫していて、そのまま運用ルールとして採用して良いレベルだと思います。
[57] [58]

整理すると、先生のルールはこうですね：

1. 変数名

- オリジナル変数名 + 「:カテゴリラベル」 + `_yes` を列名とする
- 例：`rc_agri:agriculture_yes` のような形

2. 値

- `_yes` 変数には 1/0 を入れる (1=そのカテゴリ、0=それ以外)
- `_no` 変数は基本つくらず、どうしても必要な場合だけ作る

3. 多カテゴリ

- 自動で全レベル分のダミーを残すのではなく、
 - k レベル → 必要に応じて「レベル1+2をまとめる」などを手作業で新変数に落とし込む
- つまり、「意味のある対比」だけを人手で設計したダミーにする

この設計なら：

- ・ 回帰結果を見るときに、**変数名だけで「何のカテゴリが1になっているのか」が明瞭**
- ・ 多カテゴリのごちゃごちゃした dummy が大量発生せず、「理論的に意味のある組み合わせだけ」を残せる
- ・ 将来の再現性も高く、「どの対比を見たかったのか」が変数名だけで分かる

というメリットがあります。 [59] [60]

実装面では、次のような分担が良さそうです。

- ・ ステップA：
 - 各 factor について `levels()` からカテゴリ名を取り、
 - `paste0(varname, ":", level, "_yes")` という列名で 0/1 を作る「汎用 dummy 生成関数」を用意する。 [61] [62]
- ・ ステップB：
 - その中から「残したい列だけ残す」or「レベル統合した新ダミーを手で作る」
 - これは理論・解釈の問題なので、先生が手作業でやる (do-file 的にスクリプトで明示)。

この方針をベースに、「ステップAの汎用 dummy 生成関数」を tidyverse 風に書くとしたら、どういう形だと自分のワークフローに一番乗りやすそうでしょうか？ 例えば `mutate_dummy_all(df, vars = c("rc_agri", "rc_media_tv", ...))` みたいな形が良いのか、あるいは「データフレームを渡したら factor を全部展開してくれる」関数が良いのか、イメージを教えてください。

森

`mutate_dummy_all(df, vars = c("rc_agri", "rc_media_tv", ...))`が良いです。

この関数の出力結果を踏まえ、分析用ダミー変数選択 + 複数カテゴリをまとめたダミー変数作成に移りたいと思います

了解です。その関数設計、とても扱いやすいと思います。[73] [74]

ここで押さえておきたい設計ポイントは2つです。

1. `mutate_dummy_all` の役割

イメージとしては、こんな動きになります。

- 入力:
 - `df` (大きな結合済みデータ)
 - `vars` (ダミー化したい factor 変数のベクトル)
- 出力:
 - 元の `df` + 「オリジナル変数名:カテゴリラベル_yes」形式の 0/1 変数
 - 元の factor 変数は、ここでは **まだ削除しない** (後で先生が「分析用ダミー選択 + 手作業でのレベル統合」をしたあとに削除)

二値変数の場合は、

- `factor_levels` が "Yes_No" でも "No_Yes" でも、
- `*_yes` には「Yes に対応するレベルなら 1、そうでなければ 0」を必ず入れるようにすれば、オリジナルのレベル順序に依存しない 0/1 が得られます。[75] [73]

多カテゴリ変数の場合は、

- `levels(x)` からカテゴリラベルを取り出して、
- `varname:level_yes` という列名で 0/1 を作る
という形にすれば、後で「どのレベルをまとめるか」を人間が見ても分かりやすくなります。
[76] [77]

2. 関数設計を少しでも工夫するなら

後々の作業を考えると、`mutate_dummy_all()` に以下のオプションがあると便利かもしれません。

- `drop_original = FALSE` (デフォルト)
 - ダミーを追加するだけで、元の `factor` は残す
 - 先生のように「出力を見てから、分析用ダミー変数選択 + レベル統合」に進むときに使う
- `drop_original = TRUE`
 - `_yes` 系のダミーを作った後、指定された `factor` 列を削除
 - 最終的な「分析用クリーンデータ」を作るときに使う

こうしておけば、**同じ関数を、探索フェーズと本番クリーンフェーズの両方で使い回せます。** ^[74]
^[78]

ここまでの方針でほぼ固まってきたように見えます。

あとは実装時に、`factor_levels` 情報 (`Yes_No / No_Yes / Non-Agriculture_Agriculture` など) をどう読み込むかを少し工夫するだけで、かなり汎用的なダミー生成レイヤーが作れそうです。 ^[73]

次のステップとして、一度 `mutate_dummy_all()` の中身を擬似コードレベルで一緒に設計してみるのはいかがでしょうか？ それとも、既に先生の頭の中にあるイメージをコードに落とす段階に移りたい感じでしょうか。

✱

`mutate_dummy_all()` の中身を擬似コードレベルで一緒に設計を進めたいです

では、かなり具体的に擬似コードを組んでみます。
実際の R コードにほぼそのまま落とせるレベルを目指します。

1. `mutate_dummy_all()` のインターフェース

- 引数
 - `df`: データフレーム
 - `vars`: ダミー化したい変数名ベクトル (`character`)
 - `drop_original = FALSE`: 元の `factor` を削除するかどうか
- 出力
 - `df` に新しいダミー変数列を追加したデータフレーム
 - 変数名は
 - 二値: `varname:label_yes`
 - 多カテゴリ: `varname:levelname_yes`

2.1変数用ヘルパー関数の擬似コード

まず、1つの factor から複数 dummy を作る小さい関数をイメージします。

```
# 引数:
#   df: データフレーム
#   var: 変数名 (長さ1の文字列)
# 戻り値:
#   list(df = df_拡張後, new_vars = 追加された列名ベクトル)

dummy_from_one_factor <- function(df, var) {
  x <- df[[var]]
  # 1. safety check: factor じゃなければ警告してスキップ
  if (!is.factor(x)) {
    # warning("var is not factor")
    return(list(df = df, new_vars = character(0)))
  }

  # 2. レベル取得
  lvls <- levels(x)

  # 3. バイナリかどうか判定
  is_binary <- length(lvls) == 2

  if (is_binary) {
    # 3a. 二値の場合: 「Yes 側」を決めるロジック
    #   - lvls が c("No", "Yes") / c("Yes", "No") / c("Non-Agriculture", "Agriculture")
    #   - ルール: "Yes" / "Agriculture" / "1" など「1にしたい文字列」を探す
    yes_level <- 決定ロジック(lvls)

    # 3b. ダミー作成
    new_name <- paste0(var, ":", yes_level, "_yes")
    df[[new_name]] <- ifelse(x == yes_level, 1L,
                           ifelse(is.na(x), NA_integer_, 0L))

    return(list(df = df, new_vars = new_name))
  } else {
    # 3c. 多カテゴリの場合: 各レベルごとに 0/1 列を作る
    new_names <- character(0)
    for (lev in lvls) {
      new_name <- paste0(var, ":", lev, "_yes")
      df[[new_name]] <- ifelse(x == lev, 1L,
                             ifelse(is.na(x), NA_integer_, 0L))
      new_names <- c(new_names, new_name)
    }
    return(list(df = df, new_vars = new_names))
  }
}
```

「Yes 側」の決定ロジックは、例えばこんな感じを想定しています。 [79]

```
決定ロジック <- function(lvls) {
  # 1. "Yes" を含むレベルがあればそれを採用
```

```

if (any(grepl("Yes", lvls, ignore.case = TRUE))) {
  return(lvls[grepl("Yes", lvls, ignore.case = TRUE)][^9_1])
}
# 2. "Agriculture" 等、あらかじめ定義した優先候補があれば採用
# 3. どれにも当てはまらなければ、レベル2番目などルールベースに決める
}

```

3. mutate_dummy_all() 本体の擬似コード

```

mutate_dummy_all <- function(df, vars, drop_original = FALSE) {
  all_new_vars <- character(0)

  for (v in vars) {
    res <- dummy_from_one_factor(df, v)
    df <- res$df
    all_new_vars <- c(all_new_vars, res$new_vars)

    if (drop_original) {
      df[[v]] <- NULL # 元の factor 変数を削除
    }
  }

  # 戻り値は、拡張後の df
  # (必要なら属性として new_vars を付けておくこともできる)
  # attr(df, "dummy_vars_created") <- all_new_vars

  return(df)
}

```

tidyverse 風にするなら `purrr::reduce()` や `dplyr::mutate()` を使った書き方もできますが、ロジックとしては上のような「for で素直に回す」形の方が、最初は読みやすいと思います。[\[80\]](#) [\[81\]](#)

4. この擬似コードの中で、先生に決めていただきたい点

1. 「Yes 側」の決定ルール

- "Yes_No", "No_Yes", "Non-Agriculture_Agriculture" のようなパターンに対して、
- どの文字列を「1 にしたい側」として優先的に拾うか（優先候補リスト）

2. 多カテゴリで `var:level_yes` を全部作ったあと、

- 「とりあえず全部作る → 後で選ぶ」で良いか
- それとも、最初から「指定したレベルだけ dummy 作成」できるようにしたいか

ここまでの擬似コードを見て、「ここはもう少しこうしたい」「この部分のロジックをもう少し細かく決めたい」という点はありませんか？

特に「Yes 側の決定ルール」について、先生のデータに即したキーワード（例: "Yes", "Agriculture", "Skilled", ...）を少し挙げていただけると、それを前提にしたロジックを書きやすくなります。

is_factor=trueの変数のパターン

- (1) dfで2値を取る & Yes_No/Si_Noの値を取る
- (2) dfで2値を取る & Yes_Noパターンではない
- (3) is_factorがtrueでラベルにYes_Noが含まれる(2値)が、実際には2値を取らない(NAまたは1値)
- (4) is_factorがtrueで複数レベル

(1) は問題なし

(2) は事前指定 (30変数)

添付EXCELファイルに変換した変数名とレベル指定 (new_variable_name, Yes_value, No_value)

(3) は変数として意味を持たないので削除 (40変数)

とても整理された分類だと思います。この(1)~(4)のパターンを前提にすると、mutate_dummy_all()の疑似コードに「どの変数をどう扱うか」の分岐をきれいに組み込めます。[82] [83]

ここでは、先生のルールを反映した「ロジックの流れ」だけをまとめます。

1. 前提：4パターンの扱い方

- (1) dfで2値 & factor_levels が Yes_No / Si_No
→ 通常ケースとして、自動で _yes ダミーを作成 (既存のルール) [82]
- (2) dfで2値 & Yes_Noパターンではない (事前指定30変数)
→ 添付 Excel の
new_variable_name, Yes_value, No_value を使って「どれを1にするか」を決めてダミー化。[82]
- (3) is_factor = TRUE & ラベルには Yes_No (2値) だが、実データは NA または1値のみ
→ 「情報を持たない」ので、mutate_dummy_all() 内で削除対象にしてしまう (ログだけ残す)。[82]
- (4) is_factor = TRUE & 複数レベル (3レベル以上)
→ 多カテゴリとして、各レベルごとに _yes ダミーを作っておく (後で先生が意味のある結合を手作業)。[82]

2. mutate_dummy_all() の疑似コードフロー

ステップ0：メタデータの準備

- meta データフレーム (variable_type_summary や variable_rename_propose をマージしたものを前提にする。[83] [82])
- ここに
 - variable
 - is_factor, is_binary, num_factor_levels, factor_levels

- (あれば) new_variable_name, Yes_value, No_value を入れておく。

ステップ1: 関数インターフェース

```
mutate_dummy_all <- function(df, vars, meta, drop_original = FALSE) {  
  # df: データ本体  
  # vars: ダミー化したい factor 名  
  # meta: 上記のメタ (variable, is_binary, factor_levels, new_variable_name, Yes_value, No_value)  
}
```

ステップ2: ループの中の分岐ロジック

擬似コード:

```
for (v in vars) {  
  
  # 1. メタから行を引く  
  info <- meta[meta$variable == v, ]  
  
  # 2. (3)パターンの判定  
  #   - is_factor == TRUE  
  #   - factor_levels に "Yes_No" を含む  
  #   - かつ、実データ df[[v]] が NA か 1種類の値しか持たない  
  if (条件(3)) {  
    # → この変数は意味を持たないので削除  
    df[[v]] <- NULL  
    next # 次の変数へ  
  }  
  
  # 3. 変数が本当に factor か確認  
  x <- df[[v]]  
  if (!is.factor(x)) next # 念のため  
  
  # 4. (1) or (2) or (4) の判定  
  
  if (info$is_binary == TRUE) {  
    # 4a. 2値 factor の場合  
  
    if (info$factor_levels %in% c("Yes_No", "Si_No", "No_Yes", "No_Si")) {  
      # → パターン(1)または「Yes_No系」  
      #   lvl <- levels(x) から "Yes" 側を判定  
      yes_level <- 決定ロジック_YES系(lvl, info$factor_levels)  
  
      new_name <- paste0(v, ":", yes_level, "_yes") # ここは先生案そのまま  
      df[[new_name]] <- ifelse(x == yes_level, 1L,  
                               ifelse(is.na(x), NA_integer_, 0L))  
    } else {  
      # → パターン(2): Yes_No ではない2値 (rc_agriなど)  
      #   meta の new_variable_name, Yes_value, No_value を使う  
      yes_level <- info$Yes_value  
      no_level <- info$No_value  
    }  
  }  
}
```

```

    new_name <- info$new_variable_name # 例: "rc_agri:Agriculture_yes"

    df[[new_name]] <- ifelse(x == yes_level, 1L,
                             ifelse(x == no_level, 0L, NA_integer_))
  }

} else {
  # 4b. 複数レベル → パターン(4)
  lvls <- levels(x)
  for (lev in lvls) {
    new_name <- paste0(v, ":", lev, "_yes")
    df[[new_name]] <- ifelse(x == lev, 1L,
                             ifelse(is.na(x), NA_integer_, 0L))
  }
}

# 5. 必要なら元の v を削除
if (drop_original) {
  df[[v]] <- NULL
}
}

```

3. 「Yes側」決定ロジックの例

パターン(1)用の「Yes側」判定を、メタ情報+レベル文字列で決めるイメージです。

```

決定ロジック_YES系 <- function(lvls, factor_levels_pattern) {

  # 例: factor_levels_pattern が "Yes_No" なら "Yes" を選ぶ
  if (factor_levels_pattern == "Yes_No") {
    return("Yes")
  }

  if (factor_levels_pattern == "No_Yes") {
    return("Yes")
  }

  if (factor_levels_pattern %in% c("Si_No", "No_Si")) {
    return("Si") # スペイン語 Yes
  }

  # それ以外は、一応 "Yes" を含む文字列を探す fallback
  idx <- grep("Yes|Sí|Si", lvls, ignore.case = TRUE)
  if (length(idx) >= 1) {
    return(lvls[idx[1]])
  }

  # それでも見つからなければ、とりあえず2番目レベルを返すなどの最後の手段
  return(lvls[min(2, length(lvls))])
}

```

4. (3)の削除条件の擬似コード

```
条件(3) <- function(df, v, info) {  
  
  # is_factor == TRUE & factor_levels に "Yes_No" を含む  
  if (!isTRUE(info$is_factor)) return(FALSE)  
  if (!grepl("Yes_No", info$factor_levels)) return(FALSE)  
  
  # 実データのユニーク値 (NAを除く) が 0 or 1 種類しかない  
  x <- df[[v]]  
  u <- unique(x[!is.na(x)])  
  return(length(u) <= 1)  
}
```

これで、「ラベル上は2値なのに、実際は1値かNAだけ」の変数を自動的に落とせます。^[82]

ここまでの疑似コードの流れで、先生の(1)~(4)の運用ルールはほぼそのまま実装に乗ると思います。

このステップ分解を見て、「この部分はもっと明示的にログを出したい（例：削除した(3)の変数一覧を保存したい）」など、追加したい要素はありますか？

森

目的と前提

- 目的: 分析用に「基本は numeric + バイナリ変数」、一部のキー変数 (state, group_treated など) だけ factor を残す。
- 前提:
 - SPSS → R 読み込み後に大きな df を作成。
 - is_factor = TRUE の変数について、(1)~(4) のパターンを使ってダミー化の扱いを決める。^{[84] [85]}

is_factor = TRUE 変数の4パターンと扱い

1. (1) dfで2値 & factor_levels が Yes_No / Si_No

- ふつうの二値変数として扱う。
- レベルのうち「Yes 側」(Yes, Sí 等) を 1、それ以外を 0 にする _yes ダミーを作る。
- 変数名: 元変数名:Yes_level_yes 例: rc_media_tv:Yes_yes。^{[85] [84]}

2. (2) dfで2値 & Yes_No パターンではない (二値だがラベルが別パターン)

- 対象は事前指定 (約30変数)。
- 添付 Excel variable_rename_propose.xlsx の new_variable_name, Yes_value, No_value を利用してダミー化。
- 値が Yes_value なら 1、No_value なら 0、それ以外/NA は NA。

- 変数名は `new_variable_name` を採用 (例: `rc_agri:Agriculture_yes`)。 [85]

3. (3) `is_factor = TRUE` かつラベル上は2値 (Yes_No) だが、実データは NA または1値のみ

- ユニーク値 (NA 除く) が 0 or 1 種類のもの。
- 情報量ゼロなので、ダミー化せず **変数自体を削除** (ログだけ残す)。 [85]

4. (4) `is_factor = TRUE` & 3レベル以上の多カテゴリ変数

- 各レベルごとに `_yes` ダミーを作成: `var:level_yes`。
- その後、先生が理論に基づいてレベル統合 (例: レベル1+2をまとめた新ダミー) を手作業で作成。
- 最終的に残すダミーは「意味のある対比」だけ。 [84] [85]

`mutate_dummy_all(df, vars, meta)` の役割

- 入力:
 - `df`: 結合済みの大きなデータ。
 - `vars`: ダミー化したい factor 変数名ベクトル。
 - `meta`: `variable_type_summary` と `variable_rename_propose` をマージしたメタ情報 (`is_factor`, `is_binary`, `factor_levels`, `new_variable_name`, `Yes_value`, `No_value` など)。 [84] [85]
 - `drop_original = FALSE/TRUE`: 元の factor を削除するかどうか。
- ループの中で、各 `v` について以下を行う:
 1. `meta` から `v` の情報を取得。
 2. 「(3)の削除条件」を満たせば、`df[[v]]` を削除して次へ。
 3. `is_binary` かどうかで分岐:
 - 二値: (1) または (2) のロジックで1本の `_yes` ダミーを作成。
 - 多値: (4) のロジックで、全レベル分の `var:level_yes` ダミーを作成。
 4. `drop_original = TRUE` の場合、ダミー作成後に元の factor 列 `v` を削除。

Yes 側の決定ロジック (パターン(1))

- `factor_levels` が "Yes_No", "No_Yes", "Si_No", "No_Si" などの Yes/No 系パターンかどうかを `meta` で判定。
- レベルベクトル `levels(x)` から:
 - "Yes", "Si" を含むレベルを優先的に Yes 側として選ぶ。
 - 見つからない場合はフォールバック (例: 2番目レベルを Yes とみなすなど) を明示的に決める。 [85]

ワークフローへの位置づけ

1. 年次・分野ごとに SPSS データを読み込み、1つの大きな df に結合。
2. 結合直後に `mutate_dummy_all(df, vars = factor_vars, meta = meta, drop_original = FALSE)` を実行し、全 factor のダミー候補を作成。
3. 出力 df を見ながら、
 - 分析に使うダミー変数を選択。
 - 多カテゴリダミーから、意味のあるレベル統合ダミーを手作業で作成。
4. 最終的な分析用 df を作る段階で、必要に応じて `drop_original = TRUE` で元の factor を削除し、「numeric + バイナリ + ごく少数のキー factor」だけに整理する。

この手順をコード化していくとき、まずはどの部分（(3)の削除判定／(2)の meta 連携／多カテゴリ展開など）から着手してみるつもりでしょうか？

✂

1. 全体方針（papeR + dplyr まわり）

- 目的:
 - SPSS の変数・値ラベルを R に取り込みつつ、
 - 加工や join/mutate では「ラベル属性が原因の型トラブル」を避ける。
- 方針:
 - 読み込み直後に **papeR** の `labels()` だけにラベルを集約し、haven の `label` 属性や独自クラスは基本的に削除。
 - 解析用 df は「素の numeric / factor / character + papeR の変数ラベル」という構造に統一する。 [86] [87]

2. SPSS 読み込み → papeR 変換 → 属性削除関数

2-1. 想定ワークフロー

1. `haven::read_sav()` 等で .sav を読み込む。
2. 各変数について:
 - 変数ラベル (`attr(x, "label")`) を取得。
 - 値ラベル (`attr(x, "labels")`) は、
 - カテゴリ変数なら factor のレベル、
 - 連続変数なら別メタデータ、
として整理。
3. **papeR** 形式で変数ラベルを設定: `labels(df) <- named_vector_of_labels`。

4. haven の label, labels, labelled クラスなど、解析の邪魔になる属性を一括で削除する関数を適用。 [\[88\]](#) [\[89\]](#) [\[86\]](#)

2-2. 用意しておくの良い関数イメージ

- `read_spss_clean(path)`
 - SPSS 読み込み → 変数ラベル抽出 → `labels(df)` へセット → 不要属性削除。
- `strip_all_labels(df)`
 - df 中の各列から label, labels, labelled クラスなどを削除し、「きれいな tibble」にする。

3. papeR ラベル ↔ SPSS 形式 変換関数

3-1. papeR → SPSS 形式 (書き出し用)

- やりたいこと:
 - **papeR** の `labels(df)` から変数ラベルを取り出し、`attr(x, "label")` に戻す。
 - factor (カテゴリ変数) については、`levels()` とメタデータから「コード値 ↔ ラベル」の対応を作り、`haven::labelled()` で labels 属性を付与。
 - 最終的に `haven::write_sav()` で .sav を書き出せる形にする。 [\[89\]](#) [\[88\]](#)
- 関数イメージ:
 - `papeR_to_spss(df, meta)`
 - meta に「各変数のコード値とラベルの対応」が保存されている前提で、それを使って `labelled` に戻す。

3-2. SPSS → papeR (読み込み時)

- 逆向き:
 - `spss_to_papeR(df_raw)`
 - 読み込み直後 df から変数ラベル・値ラベルを抽出し、
 - 変数ラベル → `labels(df)`
 - 値ラベル → factor レベル or メタデータに変換し、不要属性を削除して返す。

4. factor (カテゴリ変数) の扱い方針

- **papeR** の思想に合わせて:
 - 「カテゴリとして分析に使うもの」 → factor。
 - 「量的変数」 → numeric。
 - 「ID や自由記述」 → character。

- SPSS の値ラベル付き numeric は、
 - コード値 (例: 1/2) を残したければ *_code 列に分離し、
 - 分析用には factor 列 (*_factor) か、最終的には _yes ダミー列で扱う。
- 解析中は factor やダミーを使い、**papeR** のラベルは「変数ラベル」としてのみ利用 (値ラベルは factor/dummy 側に埋め込む)。^{[87][90]}

5. ラベル管理を補助する関数群の案

1. `spss_read_and_label(path)`
 - SPSS 読み込み → **papeR** 変数ラベル設定 → 不要属性削除 → 「型を整理した df」を返す。
2. `save_label_meta(df) / restore_label_meta(df, meta)`
 - 変数ラベル + 値ラベルを別メタデータとして保存し、加工後 df に再アタッチするための補助。
3. `svydesign_with_labels(df, design_args, meta)`
 - `svydesign()` で作るオブジェクトに対し、元 df のラベル (`labels()`) をコピーするラッパ。
 - 分析中も設計変数や主要アウトカムにラベルを維持できる。
4. `clean_for_analysis(df)`
 - 解析直前に、
 - 不要な factor 属性や label 属性を削除 (または `strip_all_labels`)。
 - 型を numeric / factor / character に再整理。
 - 「**papeR** ラベルだけ残したクリーン df」を作る最終レイヤー。

このまとめを新スレッドの最初に貼っておくと、続きの設計や実装の話がスムーズに進みやすいと思います。

今のところ、この関数群の中で「まず実装を進めたい」のはどの関数になりそうでしょうか？

✂

1. https://cran.r-project.org/package=papeR/vignettes/papeR_introduction.html
2. <https://rdr.io/cran/papeR/man/papeR-package.html>
3. <https://cran.r-project.org/web/packages/papeR/papeR.pdf>
4. <https://www.tilburgsciencehub.com/topics/manage-manipulate/loading/getting-started/spss-files-in-r/>
5. <https://pmc.ncbi.nlm.nih.gov/articles/PMC12547319/>
6. <https://r4ds.had.co.nz/factors.html>
7. <https://www.rdocumentation.org/packages/papeR/versions/1.0-6>
8. <https://stackoverflow.com/questions/65201230/how-to-keep-labels-of-a-factor-variable-after-transforming-it-to-numeric-in-r>
9. https://students.shu.ac.uk/lits/it/documents/pdf/analysing_data_using_spss.pdf
10. <https://www.rdocumentation.org/packages/base/versions/3.6.2/topics/factor>

11. <https://cran.r-project.org/package=papeR>
12. https://mantra.ed.ac.uk/practicals/Handling_Data_using_SPSS_workbook_NOV2024.pdf
13. <https://github.com/ykunisato/paper-r-jp>
14. https://www.sci.kanagawa-u.ac.jp/about/pdf/SJKU_2009_vol20_1.pdf
15. https://www.nakatani-foundation.jp/wp-content/uploads/annual_nakatani_30-2_re.pdf
16. <https://cran.r-project.org/web/packages/papeR/refman/papeR.html>
17. <https://r4ds.hadley.nz/factors.html>
18. <https://moldstud.com/articles/p-key-differences-between-factors-and-characters-in-r>
19. <https://stackoverflow.com/questions/60269527/when-to-use-as-numeric-and-as-factor-in-r>
20. https://search.r-project.org/CRAN/refmans/papeR/html/latex_table_fac.html
21. <https://forum.posit.co/t/using-haven-as-factor-without-changing-the-underlying-integer-codes/136447>
22. <https://larmarange.github.io/labelled/articles/labelled.html>
23. <https://stackoverflow.com/questions/76562914/difficulty-converting-factors-to-numeric-in-r>
24. <https://www.tilburgsciencehub.com/topics/manage-manipulate/loading/getting-started/spss-files-in-r/>
25. https://cran.r-project.org/web/packages/papeR/vignettes/papeR_introduction.html
26. <https://cran.r-project.org/web/packages/papeR/papeR.pdf>
27. https://rdr.io/cran/papeR/man/latex_table_fac.html
28. <https://www.rdocumentation.org/packages/papeR/versions/1.0-6>
29. https://www.youtube.com/watch?v=A5bQwQ4_BBc
30. https://rnetwork.obianom.com/packages/papeR/reference/latex_table_fac.ob
31. https://www.reddit.com/r/rprogramming/comments/q164nz/what_is_the_difference_between_a_character_and_a/
32. variable_type_summary.xlsx
33. <https://r4ds.hadley.nz/factors.html>
34. <https://cran.r-project.org/web/packages/papeR/refman/papeR.html>
35. https://cran.r-project.org/web/packages/papeR/vignettes/papeR_introduction.html
36. variable_type_summary.xlsx
37. <https://forum.posit.co/t/factors-and-dummy-variables/163713>
38. <https://www.rdocumentation.org/packages/stats/versions/3.6.2/topics/model.matrix>
39. [https://stackoverflow.com/questions/5048638/automatically-expanding-an-r factor-into-a-collection-of-1-0-indicator-variables](https://stackoverflow.com/questions/5048638/automatically-expanding-an-r-factor-into-a-collection-of-1-0-indicator-variables)
40. <https://stackoverflow.com/questions/68279721/using-model-matrix-to-create-dummy-variables>
41. https://qiita.com/daifuku_mochi2/items/b132dd5049feee6db7d0
42. <https://www.rdocumentation.org/packages/survey/versions/2.8-1/topics/svydesign>
43. <https://r-survey.r-forge.r-project.org/survey/html/svydesign.html>
44. <https://r-survey.r-forge.r-project.org/survey/example-design.html>
45. <https://www.rdocumentation.org/packages/regtools/versions/1.7.0/topics/factorsToDummies>
46. <https://cran.r-project.org/web/packages/makedummies/makedummies.pdf>
47. <https://search.r-project.org/CRAN/refmans/eatGADS/html/fac2dummies.html>

48. <https://stackoverflow.com/questions/65651392/model-matrix-turning-factors-into-dummies>

49. <https://www.r-bloggers.com/2022/02/how-to-include-all-levels-of-a-factor-variable-in-a-model-matrix-in-r/>

50. <https://stackoverflow.com/questions/43817302/how-can-i-create-dummy-variables-from-a-factor-in-r>

51. https://www.youtube.com/watch?v=SA8N1f_Ajms

52. variable_type_summary.xlsx

53. <https://stackoverflow.com/questions/68279721/using-model-matrix-to-create-dummy-variables>

54. <https://stackoverflow.com/questions/43817302/how-can-i-create-dummy-variables-from-a-factor-in-r>

55. <https://www.rdocumentation.org/packages/stats/versions/3.6.2/topics/model.matrix>

56. https://qiita.com/daifuku_mochi2/items/b132dd5049feee6db7d0

57. <https://forum.posit.co/t/factors-and-dummy-variables/163713>

58. variable_type_summary.xlsx

59. <https://miki.dk/post/2020-dummy-variables-in-r/>

60. <https://www.r-bloggers.com/2022/02/changing-the-column-names-for-model-matrix-output/>

61. https://www.youtube.com/watch?v=SA8N1f_Ajms

62. <https://cran.r-project.org/web/packages/makedummies/makedummies.pdf>

63. <https://hypatia.math.ethz.ch/pipermail/r-help/2024-June/479492.html>

64. <https://www.geeksforgeeks.org/r-language/specify-reference-factor-level-in-linear-regression-in-r/>

65. <https://rdr.io/cran/caret/man/dummyVars.html>

66. <https://stackoverflow.com/questions/18272014/renaming-of-column-names-in-model-matrix>

67. <https://statisticsglobe.com/specify-reference-factor-level-in-linear-regression-in-r>

68. https://www.reddit.com/r/rstats/comments/1esty45/multiple_regression_in_r_with_dummy_variables/

69. <https://support.bioconductor.org/p/9160184/>

70. <https://stackoverflow.com/questions/73353913/changing-the-names-of-dummy-variables-in-r>

71. <https://stackoverflow.com/questions/47234543/factor-levels-dummy-variable-r>

72. https://www.youtube.com/watch?v=f_iphKYk33Q

73. variable_type_summary.xlsx

74. <https://cran.r-project.org/web/packages/makedummies/makedummies.pdf>

75. <https://stackoverflow.com/questions/43817302/how-can-i-create-dummy-variables-from-a-factor-in-r>

76. <https://stackoverflow.com/questions/73353913/changing-the-names-of-dummy-variables-in-r>

77. <https://miki.dk/post/2020-dummy-variables-in-r/>

78. <https://rdr.io/cran/caret/man/dummyVars.html>

79. variable_type_summary.xlsx

80. <https://cran.r-project.org/web/packages/makedummies/makedummies.pdf>

81. <https://rdr.io/cran/caret/man/dummyVars.html>

82. variable_rename_propose.xlsx

83. variable_type_summary.xlsx

84. variable_type_summary.xlsx

85. variable_rename_propose.xlsx

86. https://cran.r-project.org/web/packages/papeR/vignettes/papeR_introduction.html
87. <https://cran.r-project.org/web/packages/papeR/refman/papeR.html>
88. <https://www.tilburgsciencehub.com/topics/manage-manipulate/loading/getting-started/spss-files-in-r/>
89. <https://pmc.ncbi.nlm.nih.gov/articles/PMC12547319/>
90. <https://r4ds.hadley.nz/factors.html>